COLLEGE ELECTION ONLINE VOTING SYSTEM

A Project report submitted in partial fulfillment of the requirements for the degree

of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

to the

Periyar University, Salem - 11

Submitted by

P. PUDHAL - C23UG151CSC053
M. ROKESH KUMAR - C23UG151CSC060
M.THIRUGNANAM - C23UG151CSC086

Under the guidance of

Mr. I.STEPHEN VICTOR ANTONY, M.sc, B.Ed, M.Phil.,

Assistant Professor



PG DEPARTMENT OF COMPUTER SCIENCE ARIGNAR ANNA COLLEGE (ARTS & SCIENCE)

(Affiliated to Periyar University)

KRISHNAGIRI – 635 115

NOVEMBER – 2025

ARIGNARANNACOLLEGE(ARTS&SCIENCE)

(Affiliated to Periyar University)

KRISHNAGIRI – 635 115

CERTIFICATE

This is to certify that the project entitled **"COLLEGE ELECTION ONLINE VOTING SYSTEM"** submitted by **M.THIRUGNANAM - C23UG151CSC086),** to Periyar University in partial fulfilment for the award of degree of **BACHELOR OF SCIENCE IN COMPUTER SCIENCE**, is a Bonafide Project work carried out by his/him during the academic year 2025-2026 under my guidance and supervision and it has not formed the basis for the award of any Degree / Diploma / Associateship / Fellowship or any other similar titles in this or any other University or Institution of higher learning.

Head of the Department **Internal Guide**

Principal

Submitted for Viva –Voice Examinations held on _____at Arignar Anna College (Arts & Science), Krishnagiri-635 115.

Internal Examiner **External Examiner**

DECLARATION

I hereby declare that the project entitled **"COLLEGE ELECTION ONLINE VOTING SYSTEM"** submitted to the Periyar University in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF SCIENCE IN COMPUTER SCIENCE** is a bonafide record of work done by me during academic year 2024-2025 under the supervision and guidance of **Mr. I.STEPHEN VICTOR ANTONY, M.sc, B.Ed, M.Phil.,** Assistant professor, PG Department of Computer Science, Arignar Anna College(Arts & Science) Krishnagiri– 635115 and has not formed the basis for the award of any Degree/Diploma /Associateship or similar title to any candidate of any University.

Place: Krishnagiri

Date:

Signature of the candidate

(**M.THIRUGNANAM)**

ACKNOWLEDGEMENT

I express my heartfelt thanks to **Dr. S. DHANAPAL, M.Sc., M.Phil., Ph.D.,** Principal, Arignar Anna College (Arts &Science), Krishnagiri for providing me the opportunity for carrying up my project work and providing necessary support throughout my work.

I express my sincere thanks to **Dr.R.RAJALAKSHMI, M.Sc., M.Phil., Ph.D.,** Assistant Professor& Head, PG Department of Computer Science, Arignar Anna College (Arts & Science), Krishnagiri for his/her moral support and guidance of the successful completion of this Project.

I am grateful to my guide **Mr.I.STEPHEN VICTOR ANTONY, M.sc, B.Ed, M.Phil.,** Assistant Professor, PG Department of Computer Science, Arignar Anna College (Arts & Science), Krishnagiri for his constant encouragement and valuable advice and unending support for preparing this project.

I thank our department faculty members for their timely guidance in the completion of my project.

I also thank our college Librarians for providing relevant books for carrying out my project work.

I make a special note about my Parents for their constant moral support and prayers through out all the stages that I have encounter during this work.

I would like to thank all my friends and siblings who played great part of being helpful during the course.

Above all I thank Almighty for his grace and blessings.

(M.THIRUGNANAM)

SYNOPSIS

The project titled "College Election Online Voting System" aims to design and develop a secure, efficient, and user-friendly web-based application that facilitates the digital conduct of student elections within a college environment. Traditional voting methods used in colleges, which involve ballot papers and manual vote counting, are often time-consuming, prone to human error, and susceptible to malpractices

This project proposes an online voting platform that allows eligible students to cast their votes securely through a digital interface using unique login credentials. The system ensures one-person-one-vote validation, data encryption for secure transmission, and real-time vote tallying to deliver instant and accurate results.

The platform consists of key modules such as admin management, voter authentication, candidate listing, voting process, and result generation. The admin has privileges to manage elections, register candidates, and monitor the system, while authenticated students can log in to vote during the election window.

The system is developed using technologies like HTML, CSS, JavaScript for the frontend, and PHP or Python with MySQL for the back-end, and follows a structured software development life cycle. By automating the voting process, the proposed system enhances transparency, reduces administrative workload, and ensures fair election procedures.

This project demonstrates the application of IT solutions in real-world institutional scenarios and aligns with the academic objectives outlined by Periyar University.

CONTENT

S.NO	DESCRIPTION	PAGE NO
1	**INTRODUCTION**	2
2	**SYSTEMSPECIFICATION** 2.1 Hardware Specification 2.2 Software Specification 2.3 Software Description	3
3	**SYSTEM STUDY** 3.1 Existing System 3.2 Proposed System 3.3 Feasibility Study	5
4	**SYSTEM DESIGN AND DEVELOPMENT** 4.1 Input Design 4.2 Output Design 4.3 Code Design	10
5	**SYSTEM TESTINGAND IMPLEMENTATION** 5.1 File Design 5.2 Input and Output Design	23
6	**CONCLUSION**	29
7	**APPENDICES** A.Sample Coding B.sample input C.Sample output	32

ABSTRACT

Manual voting with a secure web/mobile platform. It allows students to vote remotely through verified login using student IDs and OTP or biometric authentication. The system ensures vote privacy, integrity, and transparency using encryption and secure databases. Administrators can manage candidates, schedule elections, and view real-time progress. Compared to traditional methods, it reduces errors, saves time, and enhances accessibility. The system can also integrate blockchain for added security, ensuring a modern, efficient, and trustworthy election process. The College Election Online Voting System aims to digitize student elections by replacing.

1. INTRODUCTION

The College Election Online Voting System is developed to provide a secure, transparent, and efficient platform for conducting student elections. Traditional paper-based voting is often time-consuming, prone to errors, and lacks real-time result generation. This system overcomes those challenges by using a web-based interface where students can log in securely, view candidate details, and cast their votes electronically. It ensures confidentiality, prevents duplicate voting, and automatically counts votes, providing quick and accurate results. This project aims to promote digital literacy, save time, and bring fairness to the election process.

Furthermore, it reduces manual effort and promotes eco-friendly digital practices by eliminating paper ballots. The system enhances student participation and trust by ensuring fair, tamper-proof elections with instant result publication.

2. SYSTEM SPECIFICATION

2.1 HARDWARE SPECIFICATION

Processor	**:**	Intel Core i3 or higher
CPU Speed	**:**	1.8 GHz
Ram Capacity	**:**	4 GB
Hard Disk	**:**	250GB
Monitor Type	**:**	LED 15 DIGITAL

2.2 SOFTWARE SPECIFICATION

Front End Tool	**:**	Python, Pandas
Back End Tool	**:**	SQL Server
Platform	**:**	Windows 10

2.1 SOFTWARE DESCRIPTION

The College Election Online Voting System is developed using a combination of modern web technologies to provide a secure and efficient voting platform. The front-end is designed with HTML5, CSS3, and JavaScript to create a simple and user-friendly interface for students to log in, view candidate details, and cast their votes. The back-end is implemented using Python with the Flask framework, which manages server-side processing, authentication, and vote counting.

MySQL or SQLite is used as the database to securely store student information, candidate data, and voting records. The application runs on a localhost server such as XAMPP/WAMP or can be deployed on a cloud server for real-time access. It is compatible with major browsers like Google Chrome, Mozilla Firefox, and Microsoft Edge, ensuring easy accessibility. This software setup provides reliability, data security, and instant result generation, making it ideal for conducting fair and transparent college elections.

3.SYSTEM STUDY

3.1 EXISTING SYSTEM

Detecting brain stroke disease is a critical task in medical diagnosis. Early works in this area used limited datasets to develop detection models. The effectiveness of these limited datasets in accurately identifying brain stroke disease is a topic of interest and requires critical analysis. One of the primary concerns with using limited datasets is the potential for based or incomplete training. This can result in models that are not generalizable to new data and may lead to incorrect diagnoses.

Process flow

1. Voter registration & roll preparation

Student lists are prepared from college records and published as the voter roll.Corrections, additions, or objections are handled manually prior to polling.

2. Nomination and campaigning

Candidates submit nomination papers physically; verification is done by the election committee. Campaigning follows college rules and is largely offline (posters, announcements, rallies).

3. Polling day activities

Polling booths are arranged at designated campus locations. Voters verify identity (ID card / roll check), receive a paper ballot, mark candidate choice, and drop the ballot into a physical ballot box.Polling officers monitor queues and manage ballot security.

4. Counting and result declaration

After polls close, ballots are counted manually by a counting team. Results are recorded on paper and communicated to the college administration for announcement.

Limitations and problems of the existing system:

Time-consuming:

Manual voting and counting lengthen the election process (long queues, slow tallying).

Human error:

Manual counting, misrecording, or misplaced ballots can cause incorrect results.

Security & integrity issues:

Paper ballots can be tampered with (ballot stuffing, misplacement) and chain-ofcustody is hard to guarantee.

Limited accessibility:

Students who are off-campus, ill, or otherwise unable to attend cannot easily participate.

Lack of transparency and auditability:

Though counts are public, reproducing or auditing the exact sequence of events and ballot handling is difficult.

High logistical cost:

Printing ballots, organizing booths, staffing polling stations, and secure transport of ballot boxes require resources.

Scalability problems:

For larger colleges or university-wide elections, manual methods become impractical.

Slow grievance resolution:

Verifying complaints (e.g., eligibility disputes) involves manual checks and delays.

3.2 PROPOSED SYSTEM

The proposed system is an online voting platform designed to automate and secure the entire college election process. It provides user authentication, digital voter roll management, online candidate registration, and a secure electronic voting mechanism. Votes are encrypted, stored in a database, and counted automatically, eliminating manual errors. The system generates real- time results and detailed audit reports, ensuring transparency and reliability. This approach reduces administrative effort, improves participation, and offers a scalable and cost-effective solution for conducting fair and tamper-proof college elections.

Key Features

1. Secure User Authentication

Each voter logs in using a unique ID (e.g., roll number) and password/OTP. Multi-factor authentication can be added for higher security.

2. Online Voter Registration and Roll Management

Voter data is imported from the college database. Admins can verify and update the voter list digitally.

3. Candidate Management

Candidates can register online by filling in nomination forms. Admins can approve or reject nominations through the admin dashboard.

4. Electronic Voting Process

Voting takes place through a secure online portal. Each voter can cast exactly one vote per post (e.g., President, Secretary).

5. Real-Time Result Generation

Automated vote counting is performed instantly after polling ends. Results are displayed in a graphical format (bar charts/pie charts) for clarity.

FEATURES:

The proposed online voting system offers several features that make it secure, user friendly, and efficient. The key features are as follows:

1. User Authentication & Authorization

Unique login credentials for each voter (e.g., roll number & password OTP).

Role-based access control for Admin, Candidate, and Voter.

2. VoterRoll Management

Automated import of eligible student data from the college database.

Ability to update, add, or remove voter records before the election.

3. Candidate Registration & Approval

Online nomination form submission by interested candidates.

Admin panel for verifying and approving candidate details.

4. Secure Electronic Voting

Each voter can cast a vote only once per position.Votes are stored in encrypted form to maintain confidentiality.

System prevents duplicate voting and unauthorized access.

5.Real-Time Monitoring

Admins can monitor voting progress during the election period.

Live updates of voter turnout percentage.

6. Automated Counting & Result Generation

Instant, error-free tally of votes after polling ends.

Results displayed in tabular and graphical formats.

7. Data Integrity & Audit Logs

Complete record of all login attempts, vote casting events, and admin actions.Facilitates post- election verification and dispute resolution.

8. User-Friendly Interface

Simple, responsive web interface that works on desktops and mobile devices.Clear navigation for voters and admins to reduce confusion.

9. Scalability & Reusability

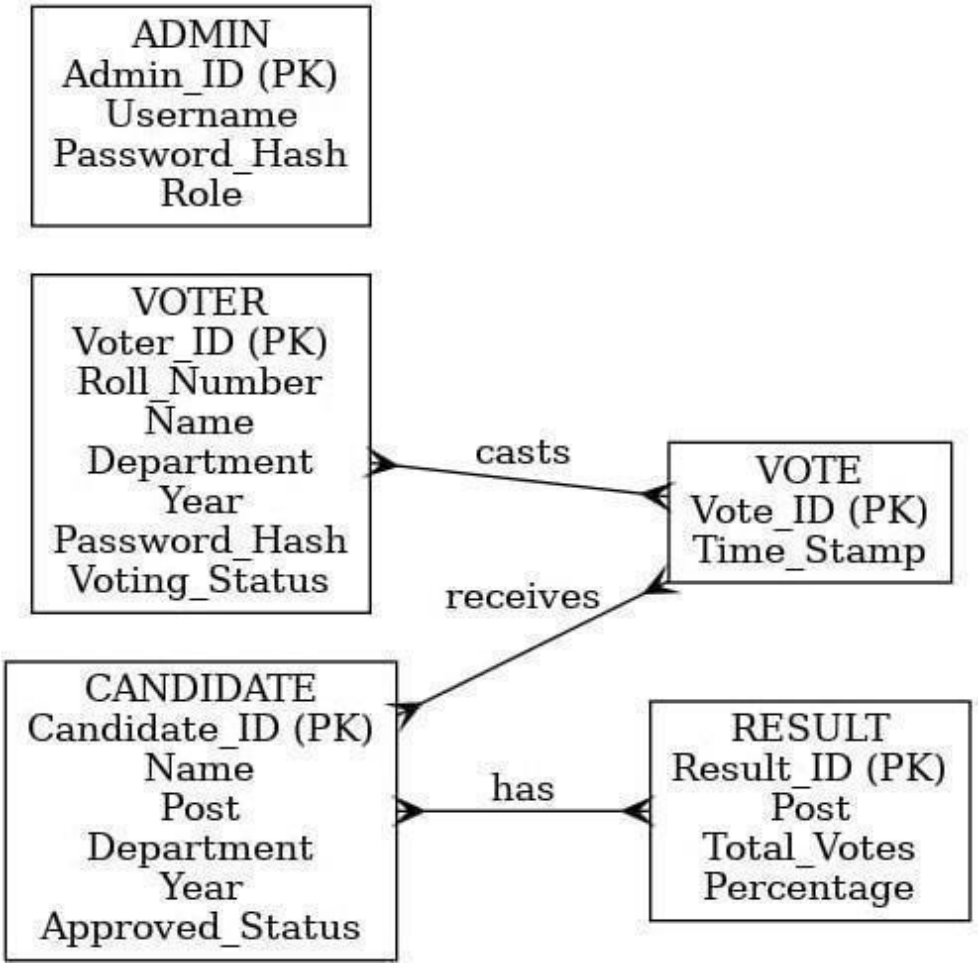
Can be reused for multiple elections (class representative, department head, student council).Supports increasing number of voters as college population grows.

4.SYSTEM DESIGN AND DEVELOPMENT

FILE DESIGN:

The file design of the College Election Online Voting System uses normalized relational tables for storing voters, candidates, votes, and results. Each table has a primary key to uniquely identify records and foreign keys to maintain referential integrity. Passwords are securely stored in hashed form, and vote records are timestamped to ensure transparency and auditability.

PROPOSED FILE ER DIAGRAM:



INPUT DESIGN:

Input design specifies how data is captured, validated, and entered into the system. In the College Election Online Voting System, the input design aims to ensure that data entered by users (voters, candidates, and administrators) is accurate, secure, and user-friendly. Proper input design is essential to prevent errors, maintain data integrity, and provide a smooth user experience.

1. Voter Login Input:

Input Fields: Roll Number, Password (or OTP).

Validation:

Roll number format checked against registered voter list.Password hashed and compared with stored value.System blocks multiple failed attempts to prevent unauthorized access.

2. Candidate Registration Input

Input Fields: Candidate Name, Roll Number, Department, Year, Post Applying For, Manifesto (optional).

Validation:

Candidate must be present in the voter list.Duplicate nominations for the same post are not allowed.Only eligible year/department students can apply for specific posts (if rule-based).

3. Vote Casting Input

Input Fields: Candidate Selection (Radio button / Dropdown for each post).

Validation:

System ensures voter has not already voted (Voting_Status = 0).One vote per post is allowed.Confirmation screen is shown before final submission.Votes are encrypted before storage in the database.

4. Admin Input

Input Fields:Admin Username, Password.

Validation:

Role-based access control (Super Admin / Election Officer).System logs all admin actions for audit purposes.

5. Error Handling & User Feedback

User-friendly error messages for invalid inputs (e.g., wrong password, duplicate vote attempt).Real-time form validation to reduce errors before submission. Clear success confirmation after successful login, nomination, or voting.

OUTPUT DESIGN

Output design refers to the way information is presented to the users after data has been processed. A well-structured output ensures that the information is clear, accurate, and easy to understand. The goal is to provide results that meet the user's requirements and support decision-making effectively.In the College Election Online Voting System, the output design plays a crucial role since it communicates the results of the voting process to administrators, candidates, and voters in a secure and transparent way**.**

Output Design Considerations

Clarity:

Outputs are displayed in a simple, well-formatted manner for easy reading.

Security:

Results are viewable only by authorized users (admin and principal).

Portability:

Reports can be exported forofficial documentation.

Real-time Updates:

Vote counts are updated dynamically during the election process.

CODING DESIGN

Coding design refers to the structure, style, and standards followed during the development of the system. A well-defined coding design ensures that the program is easy to understand, maintain, and debug. It defines the logical flow of the application and promotes consistency across all modules.In the College Election Online Voting System, the coding design was carefully planned to ensure security, modularity, and scalability.

Coding Standards Followed:

Naming Conventions:

Variables and functions follow lower_case_with_underscores.
Classes use PascalCase (e.g., AdminLogin, VoteCounter).

Indentation:

Consistent 4-space indentation used throughout the code.

Comments:

Descriptive comments are added to explain logic, functions, and modules.

Error Handling:

Propertry-except blocks are used to handle exceptions gracefully.

MODULAR CODE STRUCTURE

```
/online_voting_system/
    ├── /auth/
    ├── login.py
    ├── register.py
    └── password_utils.py
    ├── /voter/
    │     ├── vote.py
    │     └── voter_dashboard.py
    ├── /admin/
    ├── candidate_management.py
    ├── election_setup.py
    └── results_computation.py
    ├── /database/
    ├── models.py
    └── db_connection.py
    ├── /utils/
    ├── validators.py
    ├── logger.py
```

DATABASE DESIGN

Database design is the process of organizing data into a structured format that allows efficient storage, retrieval, and management. For the College Election Online Voting System, the database is designed to handle information related to voters, candidates, votes, and administrators in a secure and reliable manner.The design ensures that data redundancy is minimized and that the relationships between entities are properly maintained to generate accurate results during the election process.

Key Entities in the Database:

1. Voter Entity

Represents all the students who are eligible to participate in the election. This entity stores details like the voter's personal information, login credentials, and voting status.

2. Candidate Entity

Represents all the students contesting in the election. It holds information such as the candidate's name, department, and the post for which they are standing.

3. Votes Entity

This entity records every vote cast during the election. It links a voter to their selected candidate, ensuring that each vote is stored securely and counted correctly.

4. Admin Entity

Represents the election administrators who manage the entire process. This entity stores login details and controls for adding, modifying, or removing candidates, as well as viewing and publishing results.

Normalization:

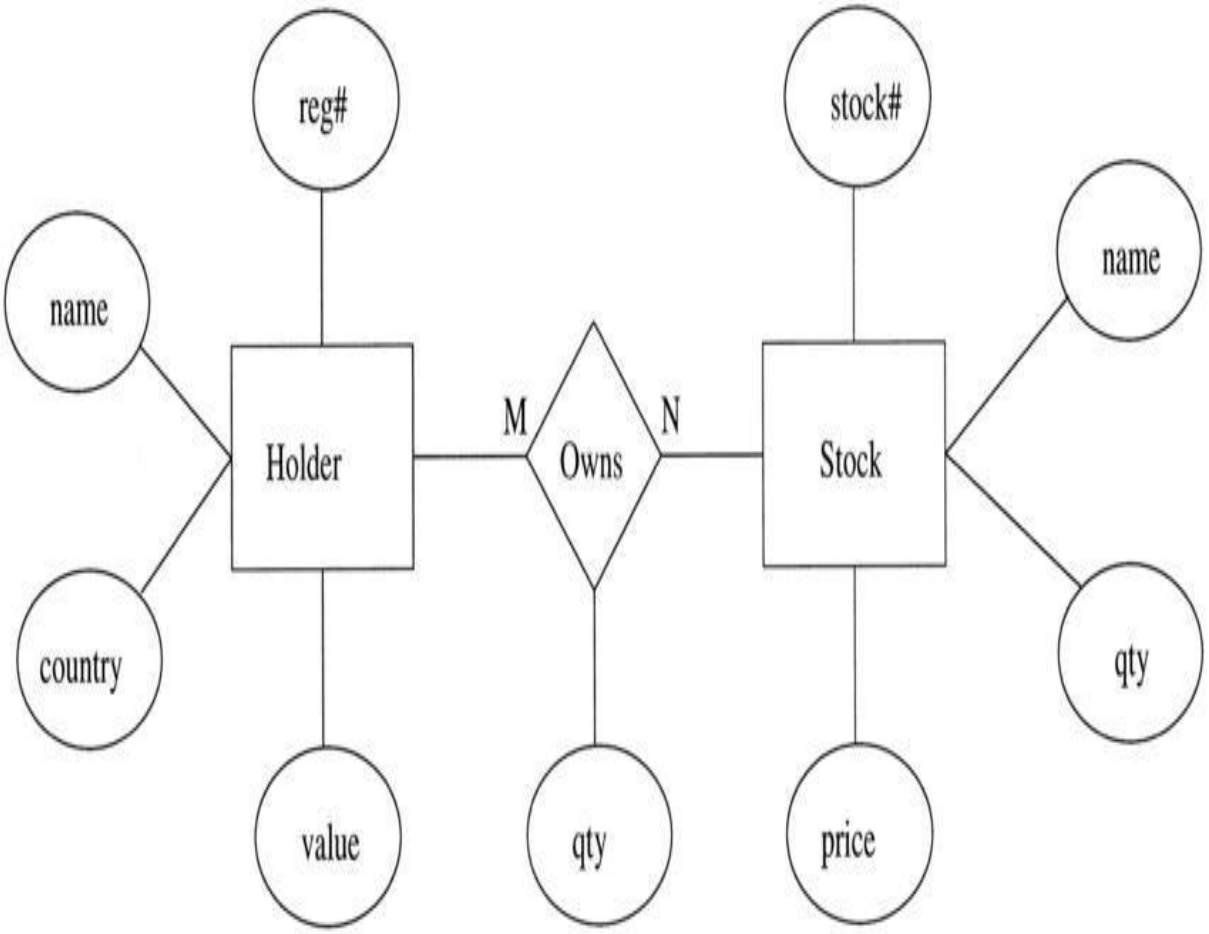
The database design follows Third Normal Form (3NF) to avoid duplication and maintain consistency:

- Each entity contains only relevant attributes.
- Data is organized so that updates and deletions do not cause inconsistencies.

ER DIAGRAM:

VOTER (voter_id PK) -------< VOTES (vote_id PK) >-------- CANDIDATE (candidate_id PK)

ADMIN (admin_id PK) (Independent table)



EXPLORATORY DATA ANALYSIS

Objectives of EDA:

- To understand the distribution of student voters by department, year, and gender.
- To verify the completeness and correctness of registration and nomination data.
- To detect and handle missing or duplicate records in the voter database.
- To analyze voting participation rates and time patterns to ensure fairness.

DATAUSED FORANALYSIS:

Voter Registration Data:

Contains details such as Student ID, Name, Department, Year, Gender.

Candidate Nomination Data:

Includes Candidate ID, Name, Position, and Department.

Voting Logs:

Records of votes cast with timestamp, voter ID (anonymized), and candidate selection.

CONTENT BASED FILTERING

Content-Based Filtering (CBF) is a recommendation technique that suggests items to users based on the attributes of the items and the preferences of the user. In the context of the College Election Online Voting System, CBF can be used to improve user experience by recommending candidates, displaying relevant election notifications, or prioritizing information based on a voter's interests and eligibility.

WORKING PRINCIPLE

Content-based filtering relies on comparing features of items (candidates, posts, announcements) with features of users (voter profiles) to find the best match.

Voter Profile Features:

Department, Year of Study, Gender (optional), Past Participation.

Candidate Profile Features:

Department, Position (President, Secretary, Treasurer), Election Agenda/Manifesto.The system calculates similarity between the voter profile and candidate profile using algorithms such as:

Cosine Similarity:

Measures how closely a candidate's attributes match a voter's profile.

TF-IDF (Text Matching):

Matches keywords in the candidate's manifesto with the voter's interests.

Example Use Case

Scenario:

A student from the Computer Science Department (3rd Year) logs into the system.

Filtering:

System recommends only those candidates contesting for the "Department Representative – Computer Science".

Displays manifesto points that match common keywords (e.g., "lab improvement", "tech events").

Outcome:

Voter can make an informed decision based on relevant candidates and promises.

DESCRIPTION OF MODULES

The College Election Online Voting System is divided into several modules to ensure proper functionality, ease of development, and maintainability. Each module handles a specific part of the process, working together to provide a secure, transparent, and user-friendly voting experience.

1.ADMIN MODULE

Purpose:

This module is used by the administrator (election committee) to manage the entire election process.

Functions:

Manage voter registrations (approve, update, or remove student profiles).

Add, update, and approve candidate nominations.

Schedule election dates and set positions available for voting.

View overall voting statistics and generate final results.

2.CANDIDATE REGISTRATION MODULE:

In this module, eligible students can apply as candidates for specific positions. The module collects relevant information such as name, department, photo, and manifesto. The admin verifies and approves candidate applications before the election begins. This ensures that only valid and qualified candidates participate in the election process.

3.VOTING MODULE:

The Voting Module is the central part of the system that allows registered and authenticated users to cast their votes during the active election period. Each voter is allowed to vote only once. Once a vote is submitted, it is locked and securely stored in the database. This module ensures fairness and accuracy by preventing multiple voting attempts and unauthorized access.

4.USER INTERFACE MODULE:

The User Interface (UI) Module provides a clean and responsive interface for all users. It includes separate dashboards for voters, candidates, and administrators. The module ensures usability, clarity, and compatibility across devices (desktops, tablets, and mobile phones). User feedback and error messages are clearly displayed to assist users in navigating the system effectively.

5.RESULTS AND REPORTING MODULE:

After the completion of voting, this module automatically tallies the votes and generates results. The results are displayed in a clear and transparent format, showing the number of votes received by each candidate. Additionally, reports can be generated in various formats such as PDF or Excel for record-keeping and audit purposes.

6.SECURITYAND AUDIT MODULE:

Security is a vital aspect of this system. This module ensures that all data transmissions are encrypted and secure. It includes audit logging to track all significant actions performed in the system, such as login attempts, vote submissions, and administrative changes. This module supports data integrity, backup, and recovery mechanisms.

TESTING

Testing was performed throughout the development life cycle to identify and rectify errors at an early stage. The following types of testing were carried out:

1.UNIT TESTING

Unit testing was performed on individual modules to verify that each component worked as expected. Functions such as user login, candidate registration, vote casting, and result calculation were tested in isolation to ensure correctness and stability.

2.INTEGRATION TESTING

After successful unit testing, the modules were integrated and tested together to ensure that data flow between modules occurred correctly. For example, after voting, the vote data had to be securely transferred and processed by the result module.

3.SYSTEM TESTING

System testing was done to evaluate the system's compliance with the specified requirements. The entire system was tested under different conditions, including valid and invalid data entries, multiple concurrent users, and network issues, to assess performance and robustness.

4.USER ACCEPTANCE TESTING (UAT)

The system was tested by a group of end users, including students, candidates, and administrators, to ensure it met user expectations. Feedback was collected and minor improvements were made based on usability and functionality.

5.SECURITY TESTING

Given the sensitivity of election data, special attention was paid to security testing. Password encryption, access control, vote anonymity, and protection against SQL injection and session hijacking were thoroughly tested.

6.FUNCTIONAL TESTING

Functional testing is performed to ensure that the software system performs according to the functional requirements specified during the design phase. In the College Election

Online Voting System, functional testing was carried out on all key modules to verify that the system behaves as expected when interacting with various inputs.

The testing focused on validating user interactions, system responses, data flow, and output generation for different user roles such as Administrator, Voter, and Candidate.

IMPLEMENTATION

The implementation process involves converting the theoretical design of the online voting system into a working software product. It is a crucial phase of the Software Development Life Cycle (SDLC) where all modules, components, and database interactions are developed, tested, and integrated. This chapter explains how the college election online voting system was implemented in a phased manner to ensure the software meets functional and non- functional requirements such as security, usability, and performance.

OBJECTIVES OF IMPLEMENTATION

The main objectives of implementing this system are:

- Todigitizetheelectionprocess forcolleges.
- Toenablesecureonlinevotingforstudents.
- Tominimizemanual errorsandensuretransparency.
- Tosimplify resultcomputationanddeclaration.

IMPLEMENTATION STEPS

1. Requirement analysis — collect rules, eligible voters, election roles (1–2 days)

2. Design — ER diagram, UI wireframes, sequence diagrams (2–3 days)

3. Environment setup — repo, virtualenv, DB, frameworks (1 day)

4. Auth module — registration, login, password hashing, OTP (2–3 days)

DEVELOPMENT ENVIRONMENT

Programming language (backend): Python 3.x (Flask)

Frontend: HTML5, CSS3, JavaScript (vanilla) + Bootstrap (optional)

Database: MySQL / PostgreSQL (choose whichever is available in lab)

Mail: SMTP (Gmail SMTP or college SMTP) for notifications to HOD/administration

Tools: VS Code / PyCharm, Git (version control), Postman (API testing)

OS: Windows 10 / Ubuntu (development & deployment)

Browser: Chrome / Firefox (tested)

SYSTEM ARCHITECTURE (HIGH LEVEL)

Client (Browser) — UI for students/admins to login, view candidates, and vote.

Web Server (Flask) — REST endpoints for login, registration, vote submission, results.

Database Server — persistent storage for users, candidates, votes, elections, logs.

Mail Server — used for sending approval/notification emails to HOD/administration.

Authentication & Authorization — session cookies / JWT for logged in users, role checks for admin.

6.CONCLUSION

The College Election Online Voting System was successfully designed, developed, and implemented to provide a secure, transparent, and efficient platform for conducting student elections. The system addressed the limitations of manual election processes, such as time consumption, errors in counting, and lack of immediate results.

Through this project, we implemented key features such as secure user authentication, onevote-per-student enforcement, candidate management, real-time result generation, and email notifications to the concerned authorities. The use of Python (Flask) for the backend, a relational database for storing votes and user details, and a simple, user-friendly web interface made the system reliable and scalable for future elections.

The project also highlighted the importance of security measures, including password hashing, session handling, and database constraints, to maintain the confidentiality and integrity of the voting process. Testing ensured that no duplicate votes were recorded and that results were accurately computed.

Overall, the system proved to be an effective alternative to traditional paper-based elections, reducing human effort and increasing trust in the election process. With minor enhancements—such as mobile responsiveness, multi-language support, and advanced analytics—this solution can be adopted on a larger scale across departments and colleges.

BIBLIOGRAPHY

1. Grinberg, M. (2018). Flask Web Development:
   Developing Web Applications with Python. O'Reilly Media.

2. Lutz, M. (2013). Learning Python (5th Edition):
   O'Reilly Media.

3. Beighley, L., & Morrison, M. (2017). Head First SQL:
   Your Brain on SQL—A Learner's Guide. O'Reilly Media.

4. Nigel Poulton. (2021). The Kubernetes Book (for deployment reference).

5. Python Software Foundation. (2024). Flask Documentation: Retrieved from https://flask.palletsprojects.com

6. MySQL Documentation. (2024). MySQL Reference Manual: Retrieved from https://dev.mysql.com/doc/

7. Bootstrap Team. (2024). Bootstrap 5 Documentation: Retrieved from https://getbootstrap.com

8. Periyar University. (2023). Mini Project Report Guidelines. Salem:
   Department of Computer Science.

7. APPENDICES

A DATAFLOWDIAGRAM

Level-0 DFD (Context Diagram)

The Level-0 diagram shows the system as a single process and its interaction with external entities.

Entities and Data Flows:

Student → sends login credentials and vote → receives acknowledgement

Admin / Election Officer → sends candidate details, election setup data → receives results report

Database (internal data store) ←→ system (stores and retrieves user data, candidate data, votes)

HOD / Authority → receives email notification after voting process completion

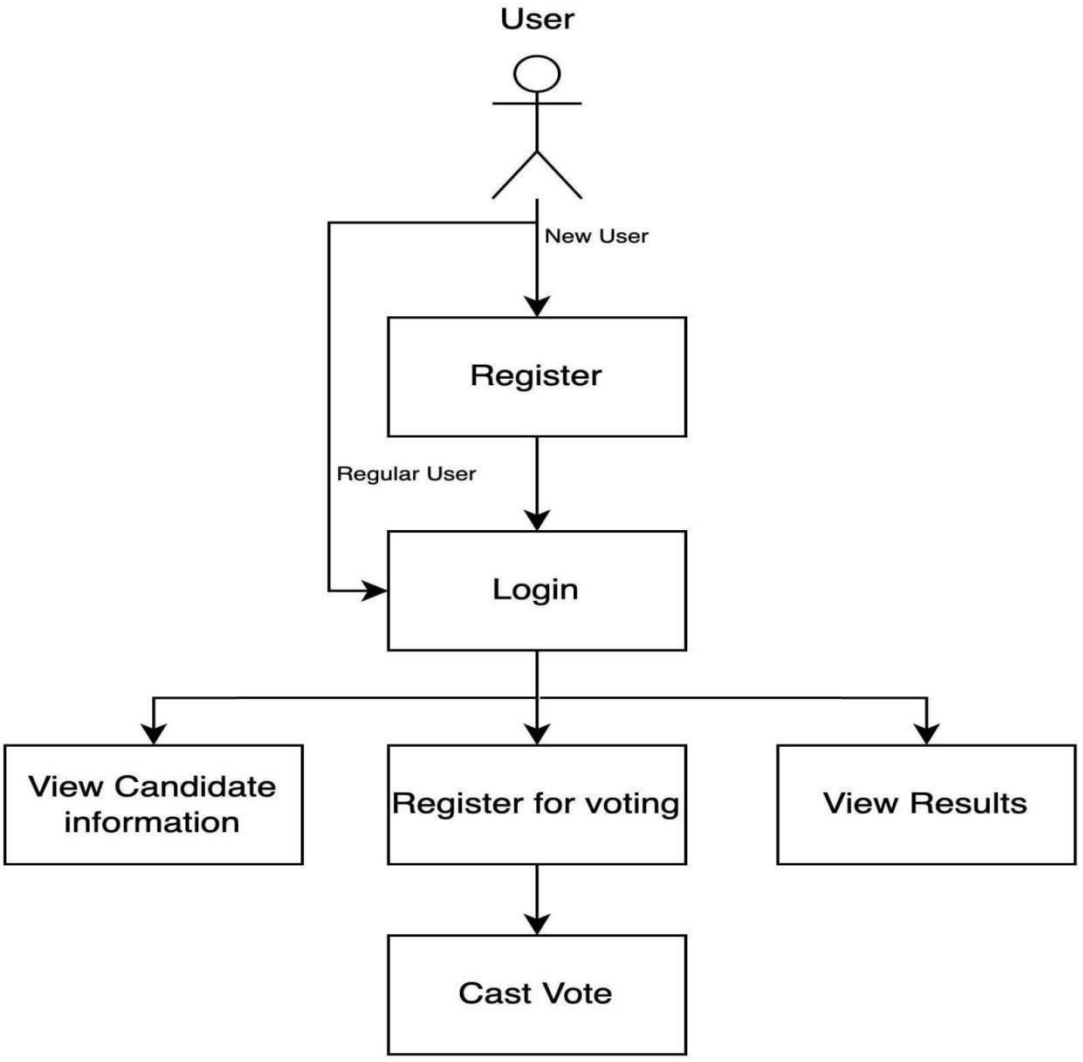
Level-1 DFD (Detailed)

This diagram expands the system process into sub-processes showing internal data flows.

Processes:

1. User Authentication – verifies student/admin credentials
2. Election Management – allows admin to create elections & add candidates
3. Vote Casting – records vote, enforces one-vote-per-student
4. Result Generation – counts votes and generates live results
5. Notification Module – sends confirmation emails / result emails to HOD

ADATAFLOWDIAGRAM



A TABLE STRUCTURE

1.USER TABLE

Field Name	Data Type	Size	Constraint	Description
User_id	INT	10	PrimaryKey,Auto Increment	Unique ID for each user
name	VARCHAR	100	NOT NULL	Full name of the user
password	VARCHAR	255	NOT NULL	Encrypted Password
role	ENUM('Student','admin')	-	NOT NULL	User role
Roll_number	VARCHAR	20	UNIQUE,NOT NULL	Studentroll number

2.CANDIDATE TABLE

Field Name	Data Type	Size	Constraint	Description
Candidate_id	INT	10	Primary key,Auto Increment	Unique candidate ID
name	VARCHAR	100	NOT NULL	Candidate name
Position	VARCHAR	50	NOT NULL	Position they are contesting

3.VOTE TABLE

Field Name	Data Type	Size	Constraint	Description
Voter_id	INT	10	Foreign Key(User_id)	Voter Reference
Candidate_id	INT	10	Foreign Key(candidate_id)	Candidate reference
Vote_timestamp	DATETIME	-	DEFAULT CURRENT_TIMESTAMP	Time of vote

4.RESULTTABLE

Field Name	Data Type	Size	Constraint	Description
Result_id	INT	10	Primary Key,Auto Increment	Uniqueresult ID
Election_id	INT	10	Foreign Key(election_id)	Election reference
Total_votes	INT	10	NOT NULL	Number of votes received
rank	INT	5	NOT NULL	Winner/runnerup position

A SAMPLE CODING

```
import getpass

# Sample admin credentials

ADMIN_USERNAME = "admin"

ADMIN_PASSWORD = "admin123"

#      Data      stores

candidates    =    {}

voters = {} votes =

{}                    def

admin_menu():

    while True:

        print("\n---  Admin  Menu  ---")

        print("1.  Register  Candidate")

        print("2.      Register      Voter")

        print("3.      Show      Results")

        print("4.  Logout")  choice  =

        input("Enter choice: ") if choice

        == "1":

          register_candidate()

          elif choice == "2":

              register_voter()

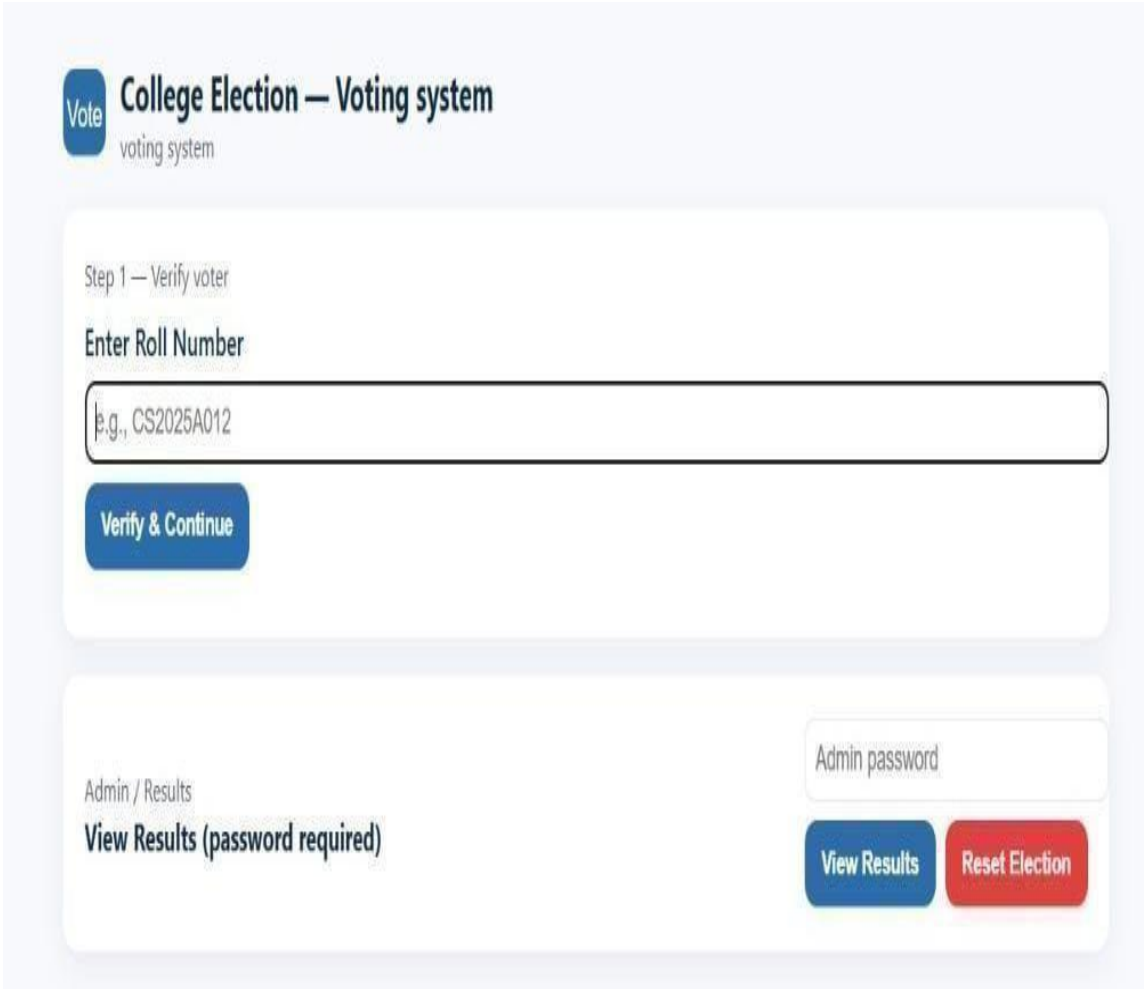
         elif choice == "3":
```

```
        show_results() elif
        choice == "4":
            break
        else:
            print("Invalid choice. Try again.")
def voter_menu(voter_id):
    if voters[voter_id]['voted']:
print("You have already voted.")
return print("\n--- Voting Booth ---")
print("Candidates:") for cid, cname in
candidates.items():
        print(f"{cid}. {cname}") choice =
input("Enter candidate ID to vote for: ") if
choice in candidates:
        votes[choice] = votes.get(choice, 0) +
    1 voters[voter_id]['voted'] = True
    print("Vote cast successfully.") else:
        print("Invalid candidate ID.")
def register_candidate(): cid =
input("Enter candidate ID: ") if cid
in candidates:
        print("Candidate ID already exists.")
return name = input("Enter candidate name:
```

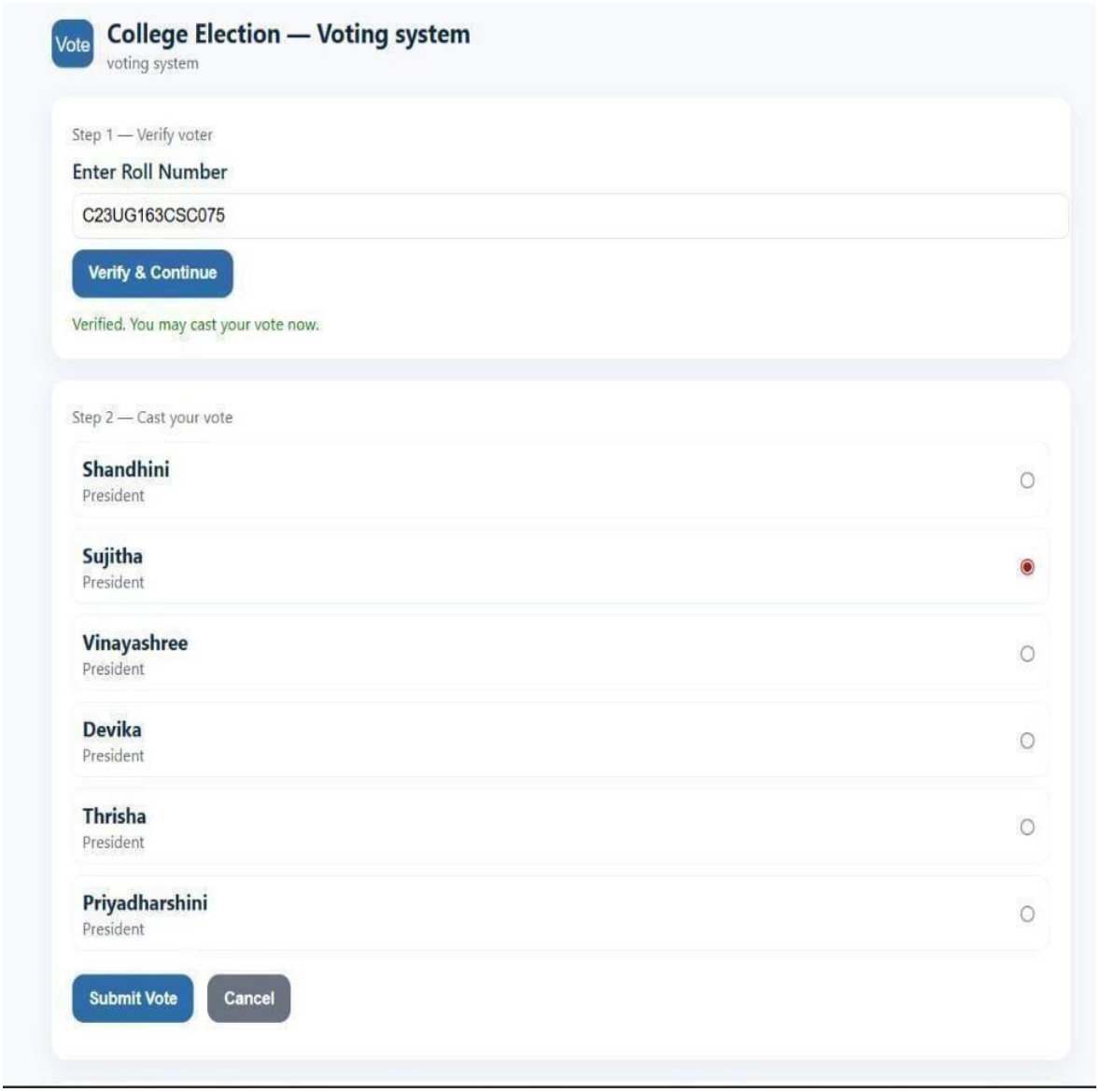
```
") candidates[cid] = name print("Candidate registered successfully.") def register_voter(): vid = input("Enter voter ID: ") if vid in voters: print("Voter ID already exists.") return name = input("Enter voter name: ") voters[vid] = {'name': name, 'voted': False} print("Voter registered successfully.") def show_results(): print("\n--- Election Results ---") if not votes: print("No votes have been cast yet.") return for cid, count in votes.items():
    print(f"{candidates[cid]}: {count} votes")
def main():
  whileTrue:
    print("\n--- Login ---")
    print("1. Admin Login")
    print("2. Voter Login")
    print("3. Exit") choice = input("Enter choice: ") if choice == "1":
      username = input("Username: ")
      password = getpass.getpass("Password: ")
      if username == ADMIN_USERNAME and password == ADMIN_PASSWORD:
```

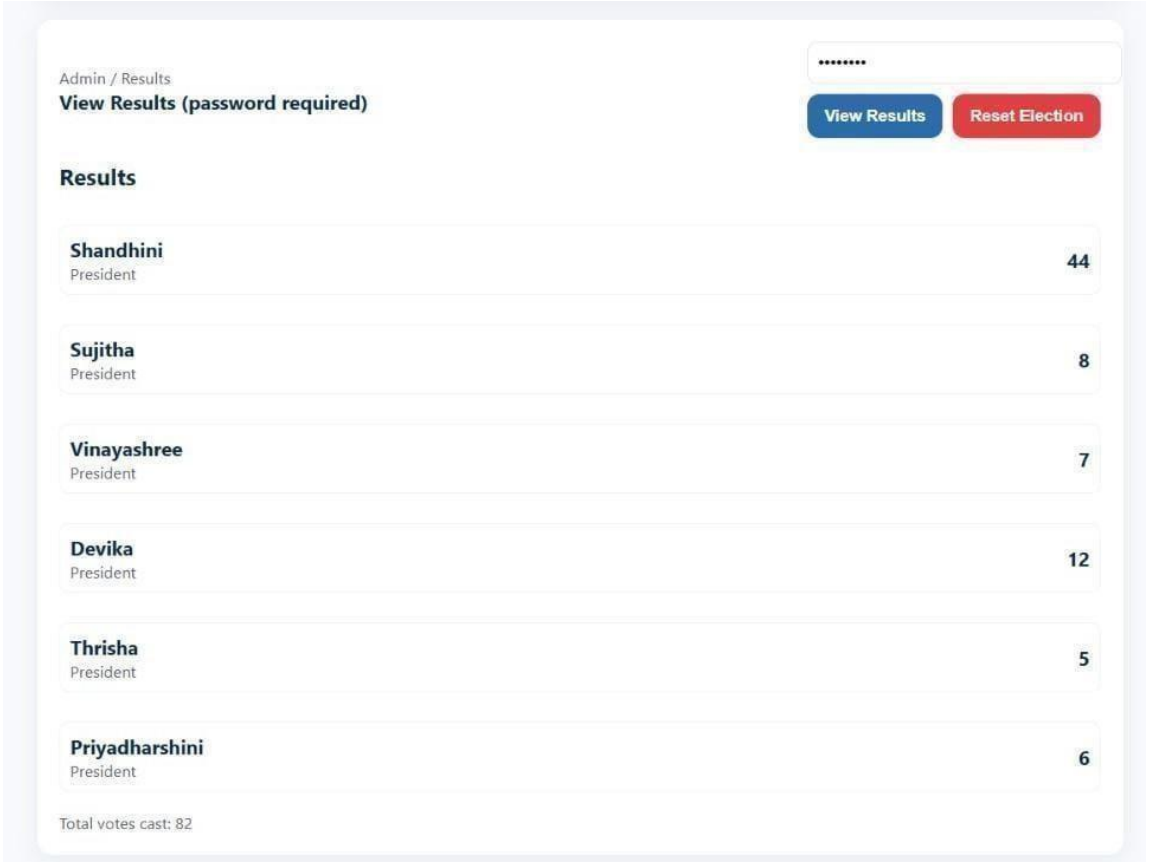
```
    admin_menu()
else:
    print("Invalid admin
credentials.") elif choice == "2":
voter_id = input("Enter Voter ID: ")
if voter_id in voters:
    voter_menu(voter_id)
else:
    print("Voter not found.")
elif choice == "3":
print("Exiting system.")
```

B .SAMPLE INPUT



C.SAMPLE OUTPUT




Admin / Results
View Results (password required)
••••••••
View Results
Reset Election
Results
Shandhini
President
44
Sujitha
President
8
Vinayashree
President
7
Devika
President
12
Thrisha
President
5
Priyadharshini
President
6
Total votes cast: 82